

SWAYAM NPTEL

ONLINE CERTIFICATION COURSE ON

Statistical and Machine Learning Methods: Basics and Applications

MODULE 09 : Machine Learning Methods

W9L1: Introduction to Neural Networks

BY

Prof. Rajib Maity

Professor, Department of Civil Engineering
Chairperson, School of Water Resources
Indian Institute of Technology Kharagpur
Kharagpur – 721302, West Bengal

CONCEPTS COVERED

- **Introduction of Neural Network**
- **Multi-Layer Perceptrons**

NPTEL

Introduction

- The human brain is a massive network of about 10^{11} interconnecting neural cells called neurons.
- It performs highly parallel computing.
- To compare, a neuron is only a very simple processor, and it is actually quite slow, in the order of 100 Hz.
- It is about ten million times slower than that of a modern computer. Still, the human brain beats the computer on many tasks involving visual interpretation, activities, sensing objects, and so on.
- Hence, the power of the brain lies not in its clock speed, nor in the computing power of a neuron, but in its massive network structure.

Introduction

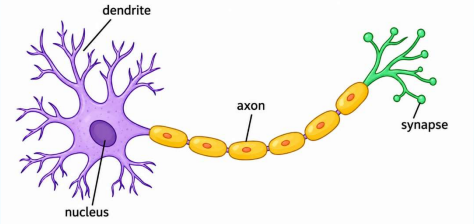
- Scientists have long been interested in understanding the computational power of large networks of interconnected neurons
- This led to the development of Neural Networks.
- Neural Networks (NN) have become dominant in Machine Learning (ML), and their use has spread from science to engineering, commerce and countless other fields.
- There are two main types of learning algorithms, supervised and unsupervised learning.
- Neural networks (NN) are more commonly used in supervised learning.

Introduction

- There are many types of neural networks, mainly utilized in various research and engineering fields.
- The feed-forward neural network represents the most widely utilized neural network architecture.
- It is characterized by unidirectional signal flow from the input layer, through intermediate layers, and finally to the output layer, devoid of any feedback connections.
- The feed-forward NN is represented by the multi layer perceptron (MLP) model.

McCulloch and Pitts Model

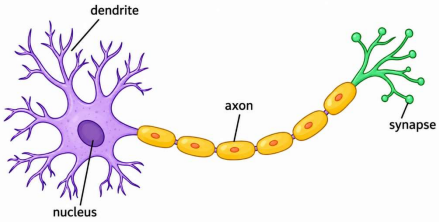
- Biological neurons are nerve cells that achieve activation by receiving and processing signals from adjacent cells.
- This foundational neurobiological concept inspired the earliest significant neural network model, developed by McCulloch and Pitts (1943).



- Their framework utilizes a binary threshold unit that aggregates a weighted sum of inputs from surrounding units; it then generates a binary output (1 or 0) predicated upon whether the calculated sum surpasses a predefined threshold.

McCulloch and Pitts Model

- This mechanism simulates a biological neuron aggregating input stimuli from adjacent cells.
- Should the cumulative strength of these signals surpass a predefined threshold, the neuron achieves activation.
- It yields an output of 1, otherwise, it remains in a resting state (yielding an output of 0).



McCulloch and Pitts Model

- For a neuron, if x_i denotes the input signal from the ith neighbour, which is weighed by a weight parameter w_i, the output of the neuron y is given by

$$y = H\left(\sum_i w_i x_i + b\right)$$

where b is called an offset or bias parameter, and H is the Heaviside step function,

$$H(z) = \begin{cases} 1, & if\ z \geq 0 \\ 0, & if\ z < 0 \end{cases}$$

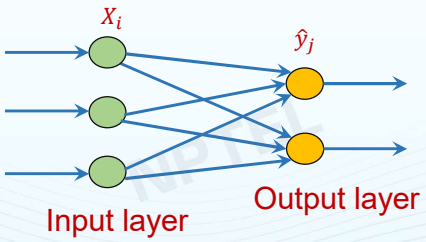
By adjusting b, the threshold level for the firing of the neuron can be changed.

McCulloch and Pitts Model

- McCulloch and Pitts established that networks comprising these artificial neurons are computationally universal, capable of executing any function a digital computer can, albeit without guaranteeing superior speed or efficiency.
- Within this framework, the neurons operate similarly to conventional logic gates.
- A significant limitation of the model, however, is the absence of a learning algorithm to determine the optimal weight and threshold parameters required for specific computational tasks.

Perceptrons

- The perceptron model consists of an input layer of neurons connected to an output layer of neurons.



Perceptrons

- Artificial neurons are frequently designated as nodes or units. A pivotal advancement in this domain was the subsequent introduction of learning algorithms, which autonomously compute the optimal weight and offset parameters required for a given computational task.
- Output from a neuron is obtained as: $\hat{y}_j = f\left(\sum_i w_{ji}x_i + b_j\right)$

where x_i denotes an input, f a specified transfer function known as an activation function, the weight parameter connecting the i-th input neuron to the j-th output neuron and b_j the offset or bias parameter of the j-th output neuron.

Perceptrons

- A more compact notation eliminates the distinction between weight and offset parameters by expressing $\sum_i w_i x_i + b$ as $\sum_i w_i x_i + w_0$, that is, b can be regarded as simply the weight w_0 for an extra constant input $x_0 = 1$, with the summation of i starting from $i = 0$ and can be written as

$$\hat{y}_j = f\left(\sum_i w_{ji}x_i\right)$$

- The step function was originally used for the activation function, but it has been replaced by other functions.

Perceptrons

- A commonly used activation function is the logistic sigmoidal function, or simply the logistic function, where 'sigmoidal' means an S-shaped function:

$$f(x) = \frac{1}{1 + e^{-x}}$$

- This function has an asymptotic value of 0 when $x \to -\infty$, and rises smoothly as x increases, approaching the asymptotic value of 1 as $x \to \infty$.

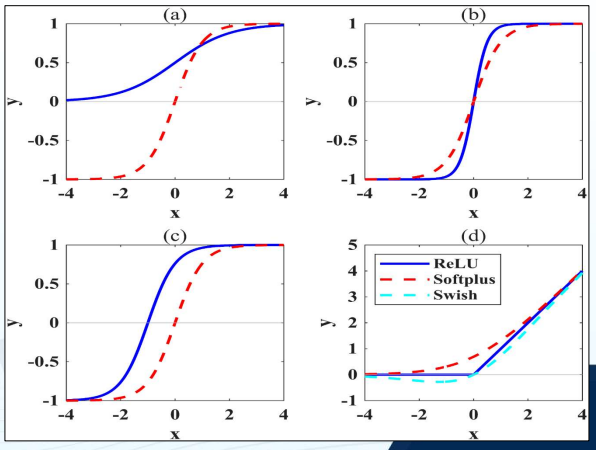
Perceptrons

Figure: a) compares the logistic sigmoid function (solid line) with the hyperbolic tangent (tanh) function (dashed).

b) Effect of the weight w in $f(wx)$ as seen by comparing $f(2x)$ (solid) and $f(x)$ (dashed) , with tanh used for f.

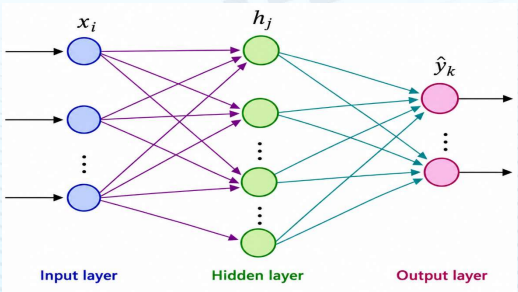
c) Effect of the offset parameter b in $f(wx + b)$ by comparing $f(x + 1)$ (solid) and $f(x)$ (dashed).

d) compares three commonly used unbounded activation functions: ReLU, Softplus, and Swish.



Multi-Layer Perceptrons (MLP)

- A Multi-Layer Perceptron (MLP) is a feed-forward neural network consisting of an input layer, one or more hidden layers, and an output layer.
- Information flows forward through the network, while the hidden layers enable the model to learn complex relationships between input and output variables.



Hidden Neurons

- An MLP can contain multiple hidden layers, where each layer receives inputs from the preceding layer; the number of hidden layers and neurons in each layer depends on the complexity of the problem.
- An MLP with one hidden layer and enough neurons can approximate any continuous function with high accuracy.
- The required number of hidden neurons depends on the complexity of the function: complex functions require more neurons, while simpler functions require fewer.
- A one-hidden-layer MLP approximates complex continuous functions by combining many simple activation functions (e.g., sigmoid or tanh).

Hidden Neurons

- This ability to combine simple non-linear functions enables an MLP to accurately model a wide range of complex non-linear relationships.
- While traditional MLPs typically used a single hidden layer, deep neural networks with multiple hidden layers can model highly complex non-linear relationships more efficiently and achieve higher prediction accuracy.
- Hidden neurons act as intermediate computational units that transform input data into meaningful output predictions.
- Although hidden neurons are generally difficult to interpret, they play a crucial role in learning complex patterns and relationships within the data.

Multi-layer Perceptrons (MLP)

- The inputs x_i $(i = 1,2, \ldots., m_1)$ are mapped to the hidden layer of neurons $h_j (j = 1,2, \ldots, m_2)$ by the equation.

$$h_j = f\left(\sum_{i=1}^{m_1} w_{ji} x_i + b_j\right),$$

And then the outputs $\hat{y}_k$ $(k = 1,2, \ldots. m_3)$ the equation is represented by

$$\hat{y}_k = g\left(\sum_{i=1}^{m_2} \hat{w}_{ki} h_j + \hat{b}_k\right),$$

where f and g are activation functions, w_{ji} and $\hat{w}_{ki}$ weight parameter matrices, and b_j and $\hat{b}_k$ are offset or bias parameters.

Multi-layer Perceptrons (MLP)

- To train the NN to learn from a dataset, we need to minimize the objective function J (also known as the loss function, cost function or error function), defined here to be one half the mean squared error (MSE) between the model output $\hat{y}_k$ and the given response data y_k,that is

$$J = \frac{1}{N}\sum_{n=1}^{N}\left[\frac{1}{2}\sum_{k=1}^{m_3}(\hat{y}_{kn} - y_{kn})^2\right],$$

where the additional subscript n indicates the nth observation, with $n = 1,2,..N$. In NN literature, a conventional scale factor of $\frac{1}{2}$ is usually included and the factor $\frac{1}{N}$ can be omitted or replaced by any positive constant.





SWAYAM NPTEL

ONLINE CERTIFICATION COURSE ON

Statistical and Machine Learning Methods: Basics and Applications

MODULE 09 : Machine Learning Methods

W9L2 : Further details on Neural Networks

BY

Prof. Rajib Maity

Professor, Department of Civil Engineering
Chairperson, School of Water Resources
Indian Institute of Technology Kharagpur
Kharagpur – 721302, West Bengal

CONCEPTS COVERED

- **Activation Functions**
- **Back-Propagation**
- **Training and Cross-training**

NPTEL

Activation Function

- Activation functions are integral components of neural networks, serving several critical purposes that enable the networks to learn and model complex relationships in data.
- It is used to compute the output values of the neurons in a neural network.
- Activation functions must be monotonic, differentiable, and quickly converging.
- It can be broadly classified into two main categories based on its characteristics:

  a) linear

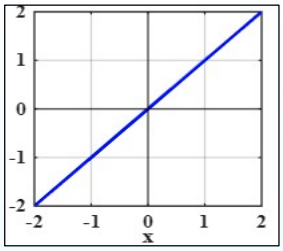
  b) non-linear

NPTEL

Activation Function

Linear Activation Function

- A linear activation function, also known as the identity or no-activation function, produces an output directly proportional to its input.
- It is mathematically expressed as: $f(x) = \alpha x$
- The linear activation function is simply a linear regression model.
- It does not create the model to be complex mapping between the network's input and output.



Linear activation function where $\alpha = 1$

NPTEL

Activation Function

Non-linear Activation Function

- Non-linear activation functions enable neural networks to learn complex and non-linear relationships between input and output variables.
- Unlike linear activation, they allow multiple hidden layers to learn different levels of information, making neural networks more effective for complex problems.
- Commonly used non-linear activation functions include Sigmoid (Logistic), Hyperbolic angent (tanh), Rectified Linear Unit (ReLU), Leaky ReLU, Swish and Softplus.

Activation Function

Non-linear Activation Function

Sigmoid Function

- The Sigmoid (Logistic) function is a non-linear activation function that transforms any real-valued input into an output between 0 and 1.
- The equation is expressed as:

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Its characteristic S-shaped curve provides a smooth transition from values close to 0 for large negative inputs to values close to 1 for large positive inputs.

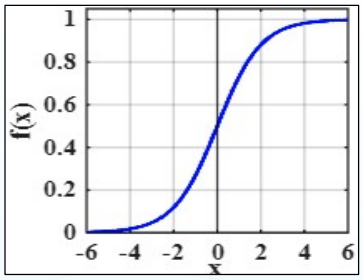
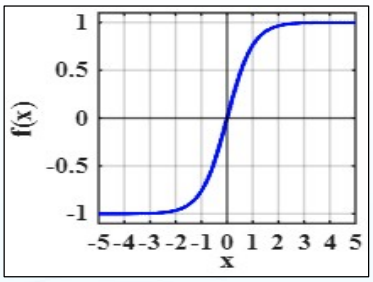


Figure: Plot of Sigmoid activation function.

Activation Function

Hyperbolic Tangent (Tanh) Function



- The hyperbolic tangent (Tanh) is a commonly used sigmoidal activation function with an S-shaped curve similar to the logistic function, but its output ranges from -1 to 1.
- The equation is expressed as:

$$f(x) = \tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

  - The Tanh function is a scaled and shifted form of the logistic function and can be expressed as :

$$\tanh(x) = 2\ logistic\ (2x) - 1$$

NPTEL

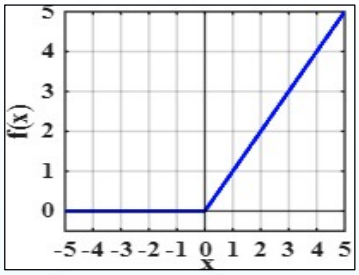
Activation Function

Hyperbolic Tangent (Tanh) Function

- Tanh is zero-centered and its outputs can be negative, zero, or positive, which removes the positive systematic bias associated with logistic outputs.
- Thus, the hyperbolic tangent (tanh) activation function is to be preferred over the logistic activation function in the hidden layer(s).
- For NN with many layers of hidden nodes, model error over test data was found to be much larger when using the logistic activation function rather than the tanh function.

Activation Function

ReLU Function



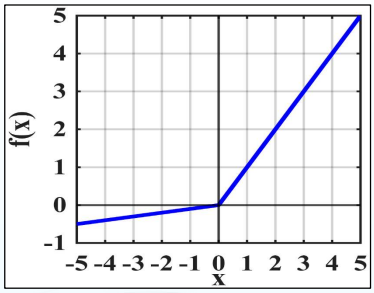
- ReLU stands for Rectified Linear Unit.
- It is a widely used non-linear activation function, particularly in neural networks and is defined as:

$$f(x) = max(0, x)$$

- ReLU produces zero output for negative inputs and passes positive inputs directly to the output, making the function simple and computationally efficient.
- For positive inputs $(x > 0)$, ReLU maintains a constant gradient of 1, allowing gradients to propagate effectively through multiple layers and reducing the vanishing-gradient problem during back-propagation.

Activation Function

Leaky ReLU Function



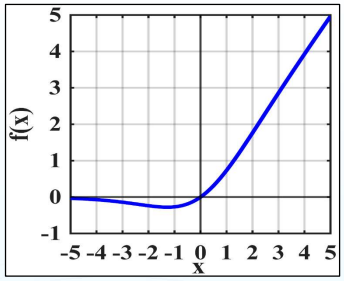
- Leaky ReLU is a modified form of the ReLU activation function designed to prevent neurons from becoming completely inactive for negative input values.
- The Leaky ReLU activation function is expressed as:

$$f(x) = \max(\alpha x, x)$$

where α is a small positive constant.

Activation Function

Swish Function



- The Swish function is a smooth, non-linear activation function that combines the input x with a sigmoid function.
- It is mathematically expressed as:

$$f(x) = x\ sigmoid\ (\beta x) = \frac{x}{1 + e^{-\beta x}}$$

where β can be simply set to the constant 1 or be treated as a tuneable hyperparameter.

- Swish function has shown strong performance in neural-network models and provides a smooth activation response for both negative and positive input values.

NPTEL

Activation Function

Softplus Function

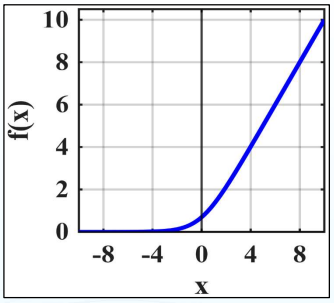


Figure: Plot of Softplus function.

- Softplus is a smooth, non-linear activation function that can be viewed as a smooth approximation of the ReLU function.
- The function is mathematically expressed as:

$$f(x) = \ln(1 + e^{x})$$

- For large positive inputs, Softplus behaves approximately like $f(x) = x$, while for large negative inputs, its output smoothly approaches zero.

NPTEL

Back-Propagation

- Back-propagation is the most popular algorithm for training of Neural Network.
- It is essentially a gradient descent technique that minimizes the network error function.
- Each input pattern is passed through the network from the input layer to the output layer, where the predicted network output is compared with the desired target output

$$E = \sum_{P} \sum_{p} (\hat{y}_i - y_i)^2$$

where y_i is observed output and $\hat{y}_i$ is model predicted output; p is number of output nodes; and P is number of training patterns.

Back-Propagation

- This error is propagated backward through the network to each node, and correspondingly the connection weights are adjusted based on equation

$$\Delta w_{ij}(n) = -\varepsilon^* \frac{\partial E}{\partial w_{ij}} + \alpha^* \Delta w_{ij}(n-1)$$

where $\Delta w_{ij}(n)$ and $\Delta w_{ij}(n-1)$ = weight increments between node i and j during the nth and $(n-1)^{th}$ pass, or epoch. A similar equation is written for correction of bias values.

- ε is the learning rate and α is the momentum.
- A learning rate is used to increase the chance of avoiding the training process being trapped in a local minima instead of global minima.

Back-Propagation

- The back-propagation algorithm consists of two main steps: a forward pass, where the input moves through the network to generate an output, followed by a backward pass, where the calculated error is propagated toward the input layer.
- During the backward pass, the network weights are adjusted using gradient descent in the direction that reduces the prediction error.
- Since gradient descent may follow a zigzag path toward the minimum, training can be slow and may sometimes become trapped in a local minimum.

Back-Propagation

- Step 0 – Initialize the network

  Initialize all weights and biases with small random values.
- Step 1 – Start the training process

  Continue the training process (step 2-9) until the specified stopping criterion is satisfied.
- Step 2 – Select training patterns

  For each input–target pair in the training dataset, perform the feed-forward and back-propagation steps.

Back-Propagation

Feed-Forward Phase

- Step 3 – Input layer

  Each input unit $(X_i = 1,2,..,n)$ receives the input x_i and passes it to all neurons in the hidden layer.

- Step 4 – Hidden layer

  Each hidden neuron $Z_j, j = 1,2,\ldots.,p)$ calculates the weighted sum of its inputs:

$$Z_j^{in} = v_j^o + \sum x_i v_j^i$$

  where v_j^i represents the connection weight and v_j^o represents the bias.

NPTEL

Back-Propagation

- Step 4 – Hidden layer

  The activation function is then applied:

$$Z_j = f(Z_j^{in})$$

  and sends this signal to output layer.

  For a sigmoid activation function,

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Step 5 – Output layer

  Each output unit $(\hat{y}_k, k = 1,2,\ldots,n)$ sums its weighted input signals

$$\hat{y}_{in}^k = w_o^k + \sum_{j=1}^{p} Z_j w_j^k \quad \text{for } j = 1,2,\ldots,p$$

Back-Propagation

Then, the activation function is applied to compute its output signal:

$$\hat{y}_k = f(Y_{in}^k)$$

where w_o^k is bias weight of the kth output neuron. It is not connected to a hidden neuron; it acts as the bias term for output neuron $\hat{y}_k$. w_j^k connection weight between the jth hidden neuron Z_j and the kth output neuron $\hat{y}_k$.

Back-Propagation

Back-propagation of Error

- Step 6 – Calculate output layer error

The network output $(\hat{y}_k,\ k = 1, 2, \dots, m)$ is compared with the corresponding target value y_k, and the output error term is calculated as:

$$\delta_k = (y_k - \hat{y}_k) f'(Y_{in}^k)$$

Now, weight and bias correction terms are calculated as

$$\Delta w_j^k = \partial \delta_k Z_j$$

$$\Delta w_o^k = \alpha \delta_k$$

Next, δ_k is sent to nodes in the previous layer.

Back-Propagation

Back-propagation of Error

- Step 7 – Calculation in the hidden layer

Each hidden node $(Z_k,\ k = 1, 2, \dots, m)$ receives the error information propagated backward from the output layer:

$$\delta_j^{in} = \sum_{k=1}^{m} \delta_k \, w_j^k \quad \text{for } k = 1,2, \dots, m$$

Then, multiplies by the derivative of its activation function hidden unit error term is then calculated as:

$$\delta_j = \delta_j^{in} f'(Z_j^{in})$$

NPTEL

Back-Propagation

Back-propagation of Error

- Step 7 – Calculate hidden layer error

The corresponding weight correction (Δv_j^i) and bias correction (v_j^o) are :

$$\Delta v_j^i = \alpha \delta_j x_i \quad \text{and} \quad \Delta v_j^o = \alpha \delta_j$$

Update Weights and Biases

- Step 8 – Update network parameters

Each output node $(\hat{Y}_k,\ k = 1, 2, \dots, m)$ updates its bias and weights $(j = 0,1, \dots, p)$

$$w_j^k(new) = w_j^k(old) + \Delta w_j^k$$

NPTEL

Back-Propagation

Update Weights and Biases

- Step 8 – Update network parameters

Each hidden node $(Z_j,\ j = 1, 2, \ldots., p)$ updates its bias and weights $(i = 0,1, \ldots, n)$

$$v_j^i(new) = v_j^i(old) + \Delta v_j^i$$

- Step 9 – Check the stopping condition
  - Check whether the predefined stopping criterion has been satisfied.
  - If not, repeat the feed-forward → error back-propagation → weight-update process until convergence.

NPTEL

Training and Cross-Training

- For effective neural network development and reliable performance assessment, the available data must be systematically organized for different stages of model training and evaluation.
- The available data set is generally partitioned into three parts –
  a)Training
  b)Cross training
  c)Validation
- The purpose of training is to determine the set of connection weights and nodal thresholds.
- It should contain sufficient patterns so that the network can mimic the underlying relationship between input and output variables adequately.

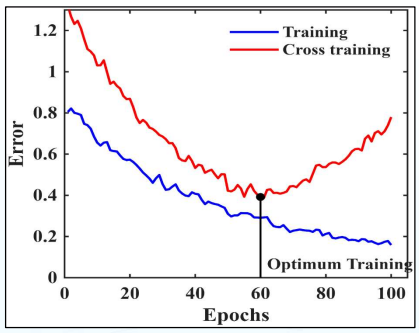
NPTEL

Training and Cross-Training

- However, there is a chance of overtraining a network, also referred as overfitting.
- Overfitting happens when the network parameters are too fine-tuned to the training data set.
- It is as if the network, in the process of trying to "learn" the underlying rule, has started trying to fit the noise component of the data as well.
- In other words, overtraining results in a network that memorizes the individual examples, rather than trends in the data set as a whole.

Training and Cross-Training

- When this happens, the network performs very well over the data set used for training, but shows poor predictive capabilities when supplied with data other than the training patterns.
- To prevent this kind of overfitting, a cross training procedure is usually recommended.
- The goal of cross training is to stop training when the network begins to be overtrained.
- The second portion of the data is reserved for this purpose.



Training and Cross-Training

- After the adjustment of network parameters with each epoch, the network is used to find the error for this data set.
- Initially, errors for both the training and cross training data sets go down.
- After an optimal amount of training has been achieved, the errors for the training set continue to decrease, but those associated with the cross training data set begin to rise.
- This is an indication that further training will likely result in the network overfitting the training data.

Model Validation

- The performance of a trained NN can be evaluated through new patterns that it has not seen during training.
- The performance of the network can be determined by computing the percentage error between predicted and desired values.
- In addition, plotting the model output versus desired response can also be used to assess NN performance.
- Since finding optimal network parameters is essentially a minimization process, it is advisable to repeat the training and validation processes several times to ensure that satisfactory results have been obtained.


FREE ONLINE EDUCATION
swayam
Thank You!
alt



SWAYAM NPTEL

ONLINE CERTIFICATION COURSE ON

Statistical and Machine Learning Methods: Basics and Applications

MODULE 09 : Machine Learning Methods

W9L3: Deep Learning

BY

Prof. Rajib Maity

Professor, Department of Civil Engineering
Chairperson, School of Water Resources
Indian Institute of Technology Kharagpur
Kharagpur – 721302, West Bengal

CONCEPTS COVERED

Convolution Neural Network

NPTEL

Deep Learning

- Deep learning (DL), a new generation of neural network-based models, has transformed industries, daily lives, and various scientific disciplines in recent years.
- Traditional Multi-Layer Perceptron (MLP) based networks generally use only one or just a few hidden layers, as deeper architectures were difficult to train effectively.
- Increasing the number of hidden layers in conventional neural networks (NN) often reduced model performance because of training challenges.

Deep Learning

- The selection of an appropriate Deep Learning model depends on
  - the characteristics of the input data
  - the problem being solved
  - the desired output
- Some of the most widely used Deep Learning models include:
- Convolutional Neural Network (CNN)
- Long Short-Term Memory Network (LSTM)
- Generative Adversarial Networks (GAN),
- Encoder-Decoder,
- U-net, etc.

Convolution Neural Network (CNN)

- CNNs are a specialized type of deep learning model designed for image-based and grid-based data, to capture spatial structures and relationships between neighboring pixels or grid cells.
- CNNs retain spatial information through local connectivity and a linear mathematical operation called convolution.
- In convolutional layers, a neuron is only connected to a small patch of neurons in the preceding layer.
- Thus, it greatly reduces the number of weights compared to the traditional fully connected layers in MLP NN models.

Convolution Operation

- In CNN, discrete convolution is performed by applying a filter, also known as kernel, F to an array A, yielding an output array B, with elements

$$B(i,j) = (A * F)(i,j) = \sum_k \sum_l A(k,l)\, F(i-k, j-l)$$

with the summation indices for all k and l

- As the convolution operation is commutative, we can write:

$$A * F = F * A$$

i.e.

$$B(i,j) = (F * A)(i,j) = \sum_k \sum_l F(k,l)\, A(i-k, j-l)$$

Convolution Operation

- It is sometimes more convenient to express the convolution operation using a correlation operator,

$$B(i,j) = (A * F)(i,j) = \sum_k \sum_l F'(k,l)\ A(i+k, j+l)$$

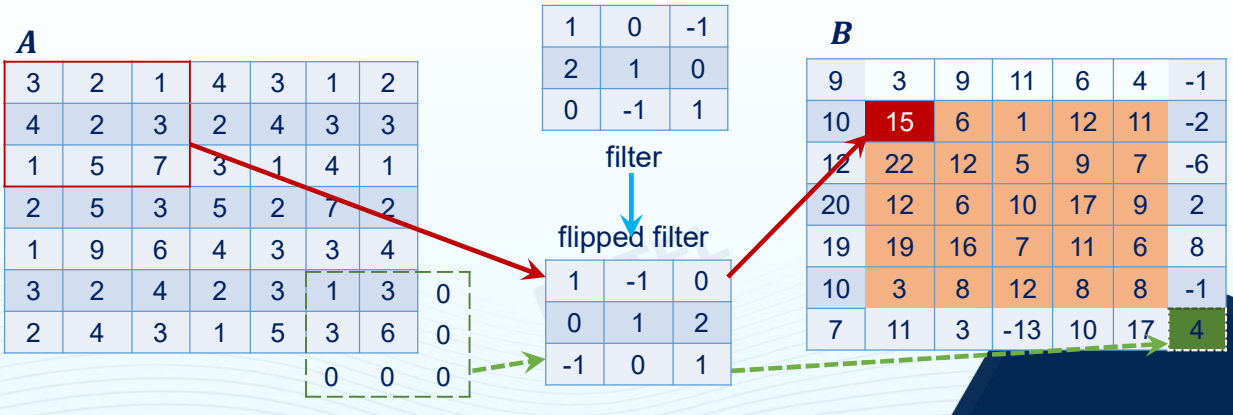
- where F' is the flipped filter or flipped kernel, obtained by reversing the order of the rows and the columns of F.
- It can be expressed as : $F' = 180^\circ$ rotation of F

For example,

$$F = \begin{pmatrix} 1 & 2 & 0 \\ -1 & 0 & 2 \\ 1 & -1 & 2 \end{pmatrix} \quad \text{and} \quad F' = \begin{pmatrix} 2 & -1 & 1 \\ 2 & 0 & -1 \\ 0 & 2 & 1 \end{pmatrix}$$



Convolution Operation



For the top left 3 × 3 patch in A, the convolution give

$$= 3 \times 1 + 2 \times (-1) + 1 \times 0 + 4 \times 0 + 2 \times 1 + 3 \times 2 + 1 \times (-1) + 5 \times 0 + 7 \times 1 = 15$$



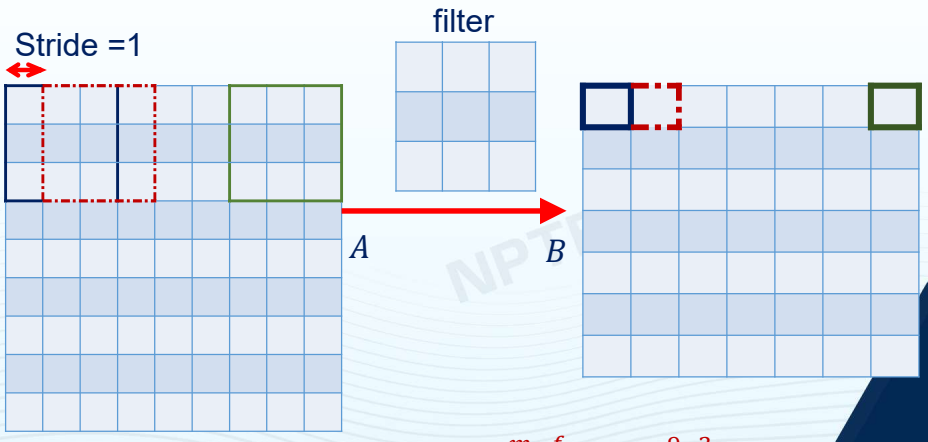
Convolution Operation

- All the elements of B are computed by shifting the flipped filter F' both horizontally and vertically across A, one grid or pixel at a time (i.e. using stride, $s = 1$), and a dot product is taken with F'.
- The resulting matrix B has a smaller size than A
  - In this example, A is 7 × 7 and B is 5 × 5 matrix.
- In general, if the width of matrix A is m, width of filter is f, then the width of B is $m - f + 1$.
- In earlier example, a 5 × 5 filter would result in a 3 × 3 output matrix (B).

Convolution Operation

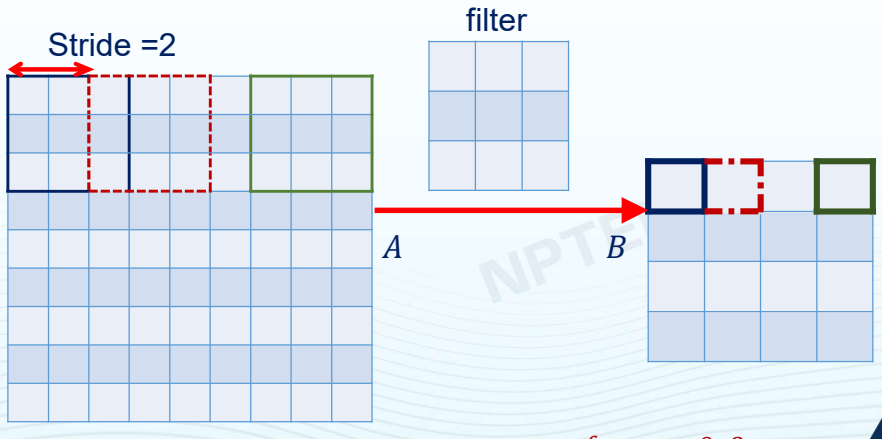
- The stride s is a hyperparameter.
- If $s = 2$ is used, the patch over A to be operated on by F' is shifted by two pixels or grids.
- In general, if the width of matrix A is m, width of filter is f, and stride is s, then the width of the output matrix B is $m - f + 1$.
- The width of the matrix B becomes $\frac{m-f}{s} + 1$, so higher values of s more the reduction in the size of the output matrix B.

Convolution Operation



For stride =1, width of the matrix $B = \frac{m-f}{s} + 1 = \frac{9-3}{1} + 1 = 7$

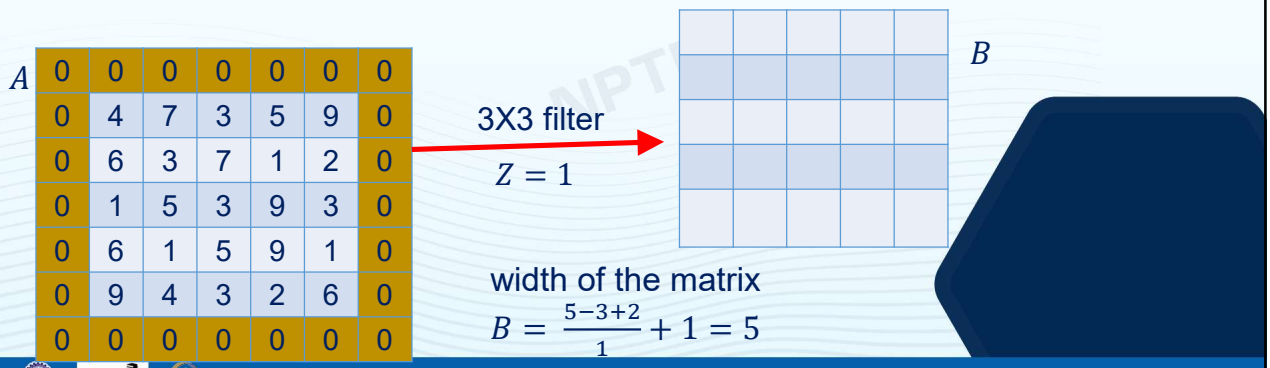
Convolution Operation



For stride =2, width of the matrix $B = \frac{m-f}{s} + 1 = \frac{9-3}{2} + 1 = 4$

Convolution Operation

- To obtain the same size output, padding with zeros beyond the boundary is needed. Then, the output matrix B will be of same size of A.
- If z number of zeros are padded beyond a boundary, the width of the matrix B is $\frac{m-f+2z}{s}+1$.



width of the matrix

$B = \frac{5-3+2}{1}+1 = 5$

Convolution Neural Network (CNN)

Pooling

- Pooling is another operation that is used to downsampling or subsampling.
- It is achieved by extracting only a single number from a patch, say $f \times f$, of the original matrix A.
- The most common pooling used is maximum pooling or max pooling, where the filter simply extracts the maximum value from the patch.
- Average pooling, which returns the mean value of the patch, was used historically.

Convolution Neural Network (CNN)

Pooling

- Generally, the *max pooling* is most commonly performed over a 2 × 2 patch, which reduces the width of A by half.
- There is no adjustable weights in this operation, and thus, not considered as a layer.
- If $s < f$ pooling patches overlap with each other. This is known as overlapping pooling. On the other, if $s = f$ in non-overlapping pooling neighbouring patches do not overlap.



NPTEL

Convolution Neural Network (CNN)

Pooling

- Similarly for average pooling



NPTEL

Key Advantages of CNN

- The reason is that CNN can use a filter with only a modest number of weights on an entire image is because the local statistics and signals in images are usually invariant to location.
- If a pattern can appear in one part of the image, it could appear elsewhere in the image.
- The same filter weights can be used for every part of the image/array.
- This allows CNN to greatly reduce the number of weights as compared to the traditional fully connected neural networks.





SWAYAM NPTEL

ONLINE CERTIFICATION COURSE ON

Statistical and Machine Learning Methods: Basics and Applications

MODULE 09 : Machine Learning Methods

W9L4: Deep Learning for Time Series

BY

Prof. Rajib Maity

Professor, Department of Civil Engineering
Chairperson, School of Water Resources
Indian Institute of Technology Kharagpur
Kharagpur – 721302, West Bengal

CONCEPTS COVERED

Deep Learning for Time Series

NPTEL

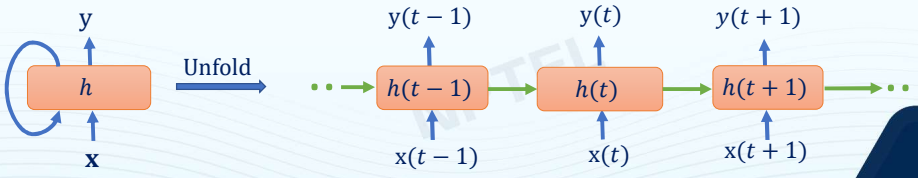
Deep Learning

- In the previous lecture, we have discussed about Convolution Neural Network (CNN). In general, CNN is used for image recognition and semantic segmentation.
- For time series, there are many alternatives:
  - Rerecurrent Neural Network (RNN)
  - Long Short-Term Memory (LSTM)
  - Convolutional LSTM (ConvLSTM)
  - Temporal Convolutional Network (TCN)

  etc.

Recurrent Neural Network (RNN)

- Recurrent Neural Network (RNN) can be considered as the predecessor of Long Short-Term Memory (LSTM) model.
- In RNN, a feedback loop is added to a Multi-Layered Perceptron (MLP) Neural Network (NN).



- It is important to pass the information from its previous state sequentially that is essential for the time-series data having memory.

Recurrent Neural Network (RNN)

- However, the memory is lost quickly in such arrangement. This can cause the influence of earlier information to gradually diminish during the learning process, making it difficult to retain important information over long sequences.
- The short-term memory storage in RNN can be improved with additional long-term memory storage.
- This leads to Long Short-Term Memory (LSTM) model as the name suggests.

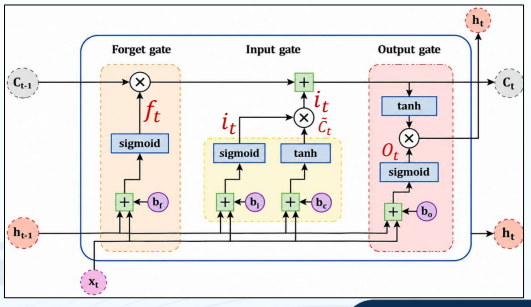
Long Short-Term Memory Network

- Thus, Long Short-Term Memory (LSTM) is an important Deep Learning model, which is designed to process sequential and time-series data.
- In LSTM, a separate cell state is incorporated that helps to preserve important information over longer time periods.
- The LSTM network consists of a sequence of LSTM cells, where each cell receives the current input and the previous hidden state.
- An LSTM cell contains three gates:
  a) Forget gate: Receives the previous time information and decides whether to clear the long-term memory (C)
  b) Input gate: Updates the long-term memory (C)
  c) Output gate: Generates the current information at time t

Long Short-Term Memory (LSTM) Network

Notations:

- x_t is the input data at time t,
- h_{t-1} is short-term information from the previous hidden state $t-1$.
- h_t is the current information at time t,
- C_{t-1} and C_t are previous and current cell state, respectively.

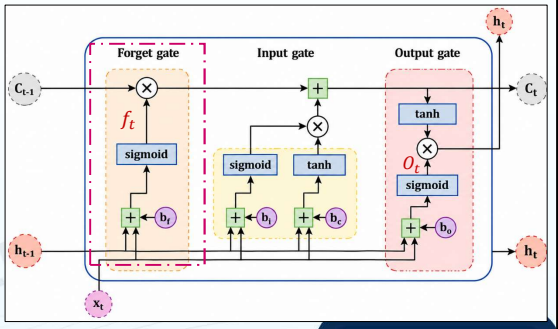


- f_t is forget gate output, i_t input gate output, O_t output gate output, $\tilde{C}_t$ candidate cell state.
- b_f forget gate bias, b_i input gate bias, b_C cell-state bias, and b_O output gate bias.
- $+$ is the addition and $\otimes$ is the multiplication

NPTEL

Long Short-Term Memory (LSTM) Network

Forget Gate



- The forget gate determines which information from the previous cell state C_{t-1} should be retained and which information should be discarded.

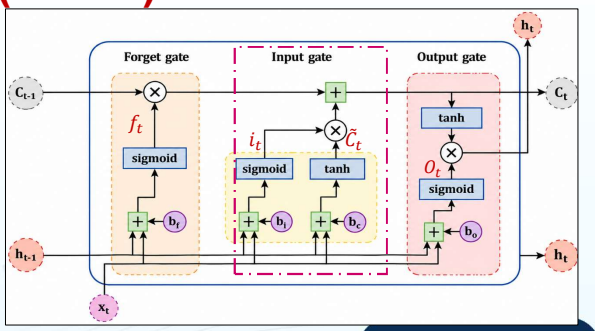
$$f_t = sigmoid\left(W_f[h_{t-1}, X_t] + b_f\right)$$

- The sigmoid function produces values between 0 and 1, where values close to 0 indicate forgetting and values close to 1 indicate retaining information.
- W_f is forget gate weights matrix.

NPTEL

Long Short-Term Memory (LSTM) Network

Input Gate



- The input gate determines which new information from the current input should be added to the cell state.

$$i_t = sigmoid(W_i[h_{t-1}, X_t] + b_i)$$

- A tanh layer generates the candidate information that can be added to the cell state:

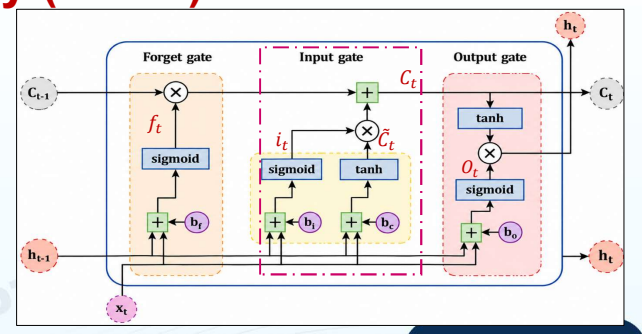
$$\tilde{C}_t = tan(W_c[h_{t-1}, X_t] + b_c)$$

where W_i is input gate weight matrix, and

W_C is candidate cell-state weight matrix.

Long Short-Term Memory (LSTM) Network

Input Gate



- The cell state is then updated by combining the retained old information with the selected new information:

$$C_t = f_t \times C_{t-1} + i_t \times \tilde{C}_t$$

- Thus, the LSTM can forget irrelevant information and store useful new information.

Long Short-Term Memory (LSTM) Network

Output Gate

- The output gate determines which information from the updated cell state should be passed to the next time step as the hidden state.

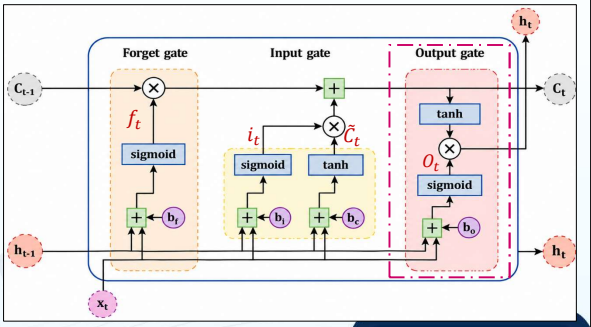
$$O_t = sigmoid(W_o[h_{t-1}, X_t] + b_o)$$

where W_O is output gate weight matrix.

- The new hidden state is obtained as:

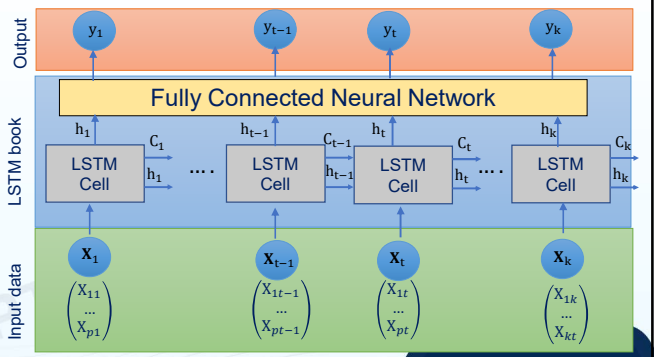
$$h_t = O_t \times tanh(C_t)$$

- Therefore, h_t carries the current information, while C_t helps retain long-term information.



Long Short-Term Memory (LSTM) Network

- The hidden state h_t generated by each LSTM cell is passed to the fully connected neural network.
- The fully connected layer combines the information from the LSTM hidden states to learn the relationship between the input sequence and the target output.



- The network processes these learned features through its connected neurons and produces the corresponding output y_t .
- Thus, the LSTM captures the temporal information, while the fully connected network converts the learned information into the final prediction.

Long Short-Term Memory (LSTM) Network

- LSTM has been applied to many fields of applications for time series prediction using external inputs and/or information from previous time steps of same time series.
- For example, streamflow prediction using meteorological data, such as precipitation, shortwave downward radiation, relative humidity and maximum/minimum temperature, etc. as inputs.





SWAYAM NPTEL

ONLINE CERTIFICATION COURSE ON

Statistical and Machine Learning Methods: Basics and Applications

MODULE 09 : Machine Learning Methods

W9L5: Hybrid Convolution and LSTM

BY

Prof. Rajib Maity

Professor, Department of Civil Engineering
Chairperson, School of Water Resources
Indian Institute of Technology Kharagpur
Kharagpur – 721302, West Bengal

CONCEPTS COVERED

Hybrid Convolution and LSTM

NPTEL

Convolutional LSTM Network

- As discussed in the previous lectures, if we have data in the form of an image, the fully connected LSTM model will end up with a very large number parameters.
- Each pixel will be considered as one input
- In such case, the *Convolutional LSTM* (ConvLSTM) will be beneficial.
- In ConvLSTM model, the fully connected layers is replaced with convolutional layers in the LSTM
- This greatly reduces the number of parameters.

Temporal Convolutional Network

- Another approach is to adapt CNN for time series problems.
- This leads to Temporal Convolutional Network (TCN).
- In 2-dimensional convolution is expressed as:

$$B(i,j) = (F * A)(i,j) = \sum_k \sum_l F(k,l)\, A(i-k, j-l)$$

- From this, the 1-D dilated causal convolution used for temporal convolution is written as:

Output: $B(i)$ — 1-D filter: F — Input: A

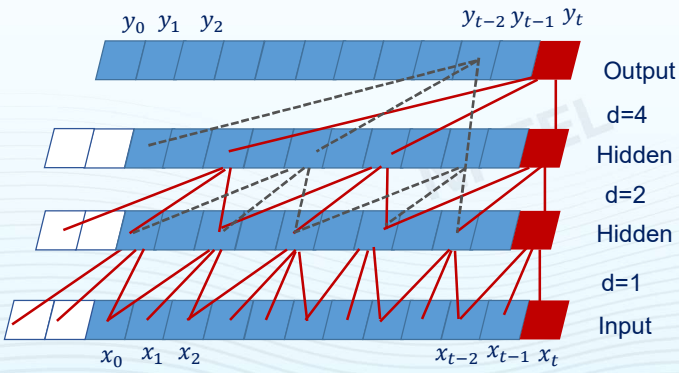
$$B(i) = (F *_d A)(i,j) = \sum_{k=0}^{f-1} F(k)\, A(i - kd)$$

Number of elements in the filter F

dilation factor (a positive integer)

Temporal Convolutional Network

- Input time series $x_0, x_1, \cdots, x_t$ passing through a dilated causal convolutional layer with $d = 1$



Temporal Convolutional Network

- A standard TCN processes an input time series through a sequence of causal convolutional layers.
- To capture patterns over time, the network uses a dilation factor that progressively grows with each successive hidden layer.
- The "receptive field" that determines how far back in time it should consider the inputs, can be expanded in two primary ways: (i) Increasing the dilation factor, or (ii) Increasing the filter size.
- By adding deeper layers with larger dilations, the network significantly expands its long-term memory capacity.
- Finally, when configuring the model, the optimal length of the input time window is typically determined by analyzing the autocorrelation or partial autocorrelation of the dataset.

Temporal Convolutional Network

- TCN may be faster as compared to RNN or LSTM based models as the concept of TCN eliminates the need of sequential processing.
- RNN or LSTM models compute step-by-step over time, forcing later predictions to wait on the completion of earlier time steps.
- TCNs use shared filters that allow for parallel convolution. As a result, TCNs can process entire, long input sequences simultaneously during both training and evaluation, speeding up computation.
- Multivariate TCN model is an extension of univariate TCN. It uses multiple input time series through separate dilated TCN sub-models.
- The output from the sub-models are concatenated into a vector, which then passes through a fully connected layer to give the final output.

